```
    {
        context.Response.OnStarting(() =>                          ❸
        {                                                          ❸
            context.Response.Headers["X-Content-Type-Options"] =   ❸
                "nosniff";                                         ❸
            return Task.CompletedTask;                             ❸
        });                                                        ❸

        await _next(context);                                      ❹
    }
}
```

❶ The RequestDelegate represents the rest of the middleware pipeline.

❷ The Invoke method is called with HttpContext when a request is received.

❸ Adds the header to the response as before

❹ Invokes the rest of the middleware pipeline. Note that you must pass in the provided HttpContext.

NOTE Using this shape approach makes the middleware more flexible. In particular, it means you can easily use DI to inject services into the `Invoke` method. This wouldn't be possible if the `Invoke` method were an overridden base class method or an interface. However, if you prefer, you can implement the `IMiddleware` interface, which defines the basic `Invoke` method.

This middleware is effectively identical to the example in listing 31.4, but it's encapsulated in a class called `HeadersMiddleware`. You can add this middleware to your app in `Startup.Configure` by calling

```
app.UseMiddleware<HeadersMiddleware>();
```

A common pattern is to create helper extension methods to make it easy to consume your extension method from Program.cs (so that IntelliSense reveals it as an option on the `WebApplication` instance). The following listing shows how you could create a simple extension method for `HeadersMiddleware`.